✅ CORRECTION — Mock DS — Practical part (on PC) — Python for Data Science

Duration: 2 h · **Total:** /20 · Documents not allowed · One notebook to submit: `NAME_Surname_DS.ipynb`

Rules — Write your code in the empty cells. Each result must be **visible** (print / display). Each cleaning decision needs a **one-line comment** explaining *why*. Answer written questions in a Markdown cell or in a comment.

Exercise	Topic	Points
1	Python basics — pharmacy stock	4
2	NumPy — daily visits	4
3	pandas — PharmaPlus sales: load, explore, clean, analyse, export	12

Files: `data/pharmacy_sales.csv` (sales of the **PharmaPlus** pharmacy chain, 2025) and `data/medicines.csv` (catalogue). In Colab, upload both files (they go to `/content`).

Exercise 1 — Python basics: pharmacy stock (4 pts)

Use only plain Python (no pandas, no NumPy).

In [1]:
```
stock = {"Doliprane": 40, "Vitamin C": 0, "Sunscreen": 12, "Cough
syrup": 4, "Thermometer": 3, "Omega 3": 7}
prices = {"Doliprane": 3.9, "Vitamin C": 6.9, "Sunscreen": 29.0, "Cough
syrup": 7.2, "Thermometer": 25.0, "Omega 3": 17.5}
```

1.1 (1 pt) With a `for` loop on `stock.items()`, print one line per product: `Doliprane -> 40 units`. Add `(OUT OF STOCK)` when the quantity is 0 and `(LOW)` when it is between 1 and 4.

In [2]:
```
for name, qty in stock.items():
    if qty == 0:                        # test 0 first (0 is also < 5)
        label = " (OUT OF STOCK)"
    elif qty < 5:
        label = " (LOW)"
    else:
        label = ""
    print(f"{name} -> {qty} units{label}")
```

```
Doliprane -> 40 units
Vitamin C -> 0 units (OUT OF STOCK)
Sunscreen -> 12 units
Cough syrup -> 4 units (LOW)
Thermometer -> 3 units (LOW)
Omega 3 -> 7 units
```

1.2 (1 pt) Write a function `stock_value(stock, prices)` that **returns** the total value of the stock (Σ quantity × price), rounded to 2 decimals. Print the result with an f-string: `Stock value: ... TND`.

In [3]:
```python
def stock_value(stock, prices):
    total = 0
    for name, qty in stock.items():
        total += qty * prices[name]
    return round(total, 2)

print(f"Stock value: {stock_value(stock, prices)} TND")   # 40*3.9 + 12*29 + 4*7.2 + 3*25 + 7*17.5 = 730.3
```

```
Stock value: 730.3 TND
```

1.3 (1.5 pt) Write a function `sell(stock, name, qty)` that:

- returns `False` and prints `Unknown product` if `name` is not in the stock;
- returns `False` and prints `Not enough stock` if there are fewer than `qty` units;
- otherwise removes `qty` units from the stock and returns `True`.

Test it with: `sell(stock, "Sunscreen", 5)`, `sell(stock, "Thermometer", 10)`, `sell(stock, "Aspirin", 1)`, then print `stock["Sunscreen"]`.

In [4]:
```python
def sell(stock, name, qty):
    if name not in stock:
        print("Unknown product")
        return False
    if stock[name] < qty:
        print("Not enough stock")
        return False
    stock[name] -= qty
    return True

print(sell(stock, "Sunscreen", 5))     # True
print(sell(stock, "Thermometer", 10))  # Not enough stock -> False
print(sell(stock, "Aspirin", 1))       # Unknown product -> False
print(stock["Sunscreen"])              # 7
```

```
True
Not enough stock
False
Unknown product
False
7
```

1.4 (0.5 pt) Build the **sorted list** of products to reorder (quantity < 5) and print it with its length.

In [5]:
```
to_reorder = sorted([name for name, qty in stock.items() if qty < 5])
print(to_reorder, len(to_reorder))     # ['Cough syrup', 'Thermometer',
'Vitamin C'] 3
```

```
['Cough syrup', 'Thermometer', 'Vitamin C'] 3
```

Exercise 2 — NumPy: daily visits (4 pts)

The pharmacy of Sousse counted its customers every day for 4 weeks (Monday → Sunday).

In [6]:
```
import numpy as np

days = np.array(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])
visits = np.array([95, 110, 102, 120, 135, 160, 70,
                   98, 115, 99, 125, 140, 172, 65,
                   101, 108, 97, 118, 150, 168, 80,
                   90, 112, 105, 130, 145, 181, 75])
```

2.1 (1 pt) Print the `shape`, `size`, `ndim` and `dtype` of `visits`. Reshape it into a 2D array `weeks` with one row per week. Print the shape of `weeks`. In a comment: what would `visits.reshape((5, 6))` do, and why?

In [7]:
```
print(visits.shape, visits.size, visits.ndim, visits.dtype)   # (28,) 28
1 int64
weeks = visits.reshape((4, 7))                                 # 4 weeks
x 7 days
print(weeks.shape)                                             # (4, 7)
# visits.reshape((5, 6)) -> ValueError: 5*6 = 30 elements but the array
has 28 (size must stay the same)
```

```
(28,) 28 1 int64
(4, 7)
```

2.2 (1 pt) Without any loop, compute: the total number of visits, the average visits **per week** (one value per week) and the average visits **per weekday** (one value per day of the week, rounded to 1 decimal).

In [8]:
```
print("Total:", visits.sum())                          # 3266
print("Average per week   :", weeks.mean(axis=1))        # axis=1 ->
along each row (week)
print("Average per weekday:", weeks.mean(axis=0).round(1))   # axis=0 ->
down each column (day)
```

```
Total: 3266
Average per week   : [113.14285714 116.28571429 117.42857143 119.71428571]
Average per weekday: [ 96.  111.2 100.8 123.2 142.5 170.2  72.5]
```

2.3 (1 pt) Find the day with the most visits over the 4 weeks: print its value, the **week number (1–4)** and the **day name**. *(Hint: `argmax()` on the 1D array, then `//` and `%`.)*

In [9]:
```
best = visits.argmax()                  # position in the 1D array -> 26
print("Max:", visits.max(), "| week", best // 7 + 1, "| day", days[best
```

```
% 7])   # 181 | week 4 | Sat
```

```
Max: 181 | week 4 | day Sat
```

2.4 (1 pt)

1. How many days had **more than 130** visits?
2. Display the names of the weekdays whose average (from 2.2) is **≥ the overall average**.
3. In a comment, give the `dtype` of `np.array([1, 2.5, True])` and of `np.array([1, "2", 3.0])`, and explain the rule.

In [10]:
```
print("Days > 130:", (visits > 130).sum())              # True counts as
1 -> 8
day_avg = weeks.mean(axis=0)
print("Weekdays >= overall average:", days[day_avg >= visits.mean()])
# ['Thu' 'Fri' 'Sat']
print(np.array([1, 2.5, True]).dtype, np.array([1, "2", 3.0]).dtype)
# float64 and <U32: an array has ONE type -> type coercion to the most
general type present
# (bool -> int -> float -> str). With a string everything becomes text.
```

```
Days > 130: 8
Weekdays >= overall average: ['Thu' 'Fri' 'Sat']
float64 <U32
```

Exercise 3 — pandas: PharmaPlus sales (12 pts)

Scenario. PharmaPlus exports the sales of its 8 pharmacies every night. The finance team needs a **trusted** dataset and a few KPIs. The file is dirty. Never modify the raw DataFrame: work on a copy.

`pharmacy_sales.csv` columns: `sale_id, sale_date, pharmacy_city, med_id, quantity, unit_price, payment, customer_age, prescription` `medicines.csv` columns: `med_id, med_name, category, cost_price, list_price`

Business rules given by the manager

- One sale line contains between **1 and 50** units. Anything else is a data-entry error.
- A customer is between **18 and 100** years old. Other ages are errors → treat them as missing.
- Every sold `med_id` must exist in the catalogue.

3.1 Load and explore (2 pts)

1. Import pandas, load the two files into `sales_raw` and `meds`.
2. Display the first 5 rows, the shape, `info()`, `describe()` and the number of unique values per column of `sales_raw`.

3. Display the distinct values of `pharmacy_city` and the frequencies of `quantity`.
4. In a Markdown cell, list **at least 4** data-quality problems you see.

In [11]:
```
import os
import pandas as pd

pd.options.display.float_format = "{:,.2f}".format
DATA = "data" if os.path.exists("data/pharmacy_sales.csv") else
"/content"
sales_raw = pd.read_csv(f"{DATA}/pharmacy_sales.csv")
meds = pd.read_csv(f"{DATA}/medicines.csv")

display(sales_raw.head())
print("Shape:", sales_raw.shape)
sales_raw.info()
```

	sale_id	sale_date	pharmacy_city	med_id	quantity	unit_price	payment	custom
0	S00195	2025-12-29	Sfax	M019	3.00	25.23	Card	
1	S00670	2025-07-25	Sousse	M018	3.00	28.02	Cash	
2	S00750	2025-09-20	Tunis	M024	1.00	6.25	Cash	
3	S00603	2025-01-02	Gabès	M012	3.00	7.27	Cash	
4	S01121	2025-09-25	Monastir	M006	3.00	15.93	Cash	

```
Shape: (2440, 9)
<class 'pandas.DataFrame'>
RangeIndex: 2440 entries, 0 to 2439
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   sale_id        2440 non-null   str
 1   sale_date      2438 non-null   str
 2   pharmacy_city  2440 non-null   str
 3   med_id         2440 non-null   str
 4   quantity       2440 non-null   float64
 5   unit_price     2403 non-null   float64
 6   payment        2392 non-null   str
 7   customer_age   2380 non-null   float64
 8   prescription   2440 non-null   str
dtypes: float64(3), str(6)
memory usage: 171.7 KB
```

In [12]:
```
display(sales_raw.describe())
print(sales_raw.nunique())
print(sales_raw["pharmacy_city"].nunique(), "spellings:",
sales_raw["pharmacy_city"].unique())
print(sales_raw["quantity"].value_counts().sort_index())
```

	quantity	unit_price	customer_age
count	2,440.00	2,403.00	2,380.00
mean	3.13	19.73	51.08
std	22.10	23.47	20.15
min	-2.00	2.81	3.00
25%	1.00	6.00	34.00
50%	2.00	12.90	51.00
75%	3.00	25.15	68.00
max	999.00	122.55	210.00

```
sale_id          2400
sale_date         368
pharmacy_city      48
med_id             25
quantity            9
unit_price       1226
payment             3
customer_age       70
prescription        2
dtype: int64
48 spellings: <StringArray>
[       'Sfax',      'Sousse',       'Tunis',       'Gabès',
'Monastir',
     'Bizerte',      'Nabeul',      'Ariana',    '  GABÈS ',
'MONASTIR',
        'SFAX',      'Tunis ',       'tunis',       'Sfax ',     'Ariana
',
    'Bizerte ',      'nabeul',       ' Sfax',    '  TUNIS ',   '
Monastir',
     ' Ariana',     'BIZERTE',     'Nabeul ',      ' Gabès',
'gabès',
   '  NABEUL ',      'Gabès ',     '  SFAX ',      'ariana',    '
Bizerte',
      ' Tunis',   'Monastir ',       'TUNIS',  '  BIZERTE ', '  MONASTIR
',
      'SOUSSE',      'ARIANA',   '  SOUSSE ',      'NABEUL',
'sfax',
    'monastir',     'Sousse ',     ' Sousse',     ' Nabeul',
'bizerte',
      'sousse',       'GABÈS',   '  ARIANA ']
Length: 48, dtype: str
quantity
-2.00       1
0.00        5
1.00      862
2.00      638
3.00      458
4.00      213
5.00      257
200.00      5
999.00      1
Name: count, dtype: int64
```

✅ **Problems found:** (1) **40 duplicated rows**; (2) `pharmacy_city` has **48 spellings** for 8 cities (spaces, UPPER/lower case); (3) `quantity` contains impossible values (**−2, 0, 200, 999**) and is `float64` instead of int; (4) missing values in `unit_price`, `payment`, `customer_age`, `sale_date`; (5) `customer_age` min = 3 and max = 210 (impossible); (6) `sale_date` is text and contains invalid dates (`2025-13-05`, `unknown`, `2025-02-30`) not detected by `info()`; (7) `med_id` `M099` does not exist in the catalogue.

3.2 Clean (5 pts) — create `sales` = a cleaned copy of `sales_raw`

Do the steps **in this order**, print the number of rows after each step that removes rows, and comment every decision.

1. Compute the **percentage of missing values per column** (sorted descending).
2. Remove the exact duplicates.
3. Clean `pharmacy_city` so that exactly **8 cities** remain.
4. Convert `sale_date` to datetime (invalid dates → missing) and drop the sales without a valid date. *Why drop instead of fill?*
5. Apply the quantity rule, then convert `quantity` to `int64`.
6. Remove the sales whose `med_id` is not in the catalogue.
7. Ages outside [18, 100] become missing; then fill all missing ages with the **median** age. *Why the median and not the mean?*
8. Fill missing `payment` with `"Unknown"`.
9. Fill missing `unit_price` with the **median price of the same medicine** (`groupby` + `transform`).
10. Validate with `assert` that no missing value is left, and print a cleaning report: raw rows, clean rows, % removed.

In [13]:
```
sales = sales_raw.copy()                          # keep the raw data untouched
(sales.isna().mean() * 100).sort_values(ascending=False).round(2)
```

Out[13]:
```
customer_age    2.46
payment         1.97
unit_price      1.52
sale_date       0.08
sale_id         0.00
quantity        0.00
med_id          0.00
pharmacy_city   0.00
prescription    0.00
dtype: float64
```

In [14]:
```
print("Duplicates:", sales.duplicated().sum())
sales = sales.drop_duplicates()                   # exact copies = same sale counted twice
print("Rows:", len(sales))
```

```
Duplicates: 40
Rows: 2400
```

In [15]:
```
sales["pharmacy_city"] = sales["pharmacy_city"].str.strip().str.title()
# spaces + case
print(sales["pharmacy_city"].nunique(),
sorted(sales["pharmacy_city"].unique()))
```

```
8 ['Ariana', 'Bizerte', 'Gabès', 'Monastir', 'Nabeul', 'Sfax', 'Sousse',
'Tunis']
```

In [16]:
```
sales["sale_date"] = pd.to_datetime(sales["sale_date"], errors="coerce")
# invalid -> NaT
print("Invalid / missing dates:", sales["sale_date"].isna().sum())
sales = sales.dropna(subset=["sale_date"])      # key field: a sale
without date cannot be placed in time; inventing one would be fake data
print("Rows:", len(sales), "|", sales["sale_date"].dtype)
```

```
Invalid / missing dates: 14
Rows: 2386 | datetime64[us]
```

In [17]:
```
print("Out of the rule:", (~sales["quantity"].between(1, 50)).sum())
sales = sales[sales["quantity"].between(1, 50)].copy()   # business
rule: 1..50 units per line
sales["quantity"] = sales["quantity"].astype("int64")
print("Rows:", len(sales), "| max quantity:", sales["quantity"].max())
```

```
Out of the rule: 12
Rows: 2374 | max quantity: 5
```

In [18]:
```
unknown = ~sales["med_id"].isin(meds["med_id"])
print("Unknown med_id:", sales.loc[unknown, "med_id"].unique(),
unknown.sum())
sales = sales[~unknown].copy()                  # no cost / category ->
cannot be analysed or verified
print("Rows:", len(sales))
```

```
Unknown med_id: <StringArray>
['M099']
Length: 1, dtype: str 5
Rows: 2369
```

In [19]:
```
bad_age = (sales["customer_age"] < 18) | (sales["customer_age"] > 100)
print("Impossible ages:", bad_age.sum())
sales.loc[bad_age, "customer_age"] = np.nan                    # error
-> missing (keep the sale!)
median_age = sales["customer_age"].median()
sales["customer_age"] = sales["customer_age"].fillna(median_age)  #
median: robust to extreme values
print("Median age used:", median_age)
```

```
Impossible ages: 6
Median age used: 50.5
```

In [20]:
```
sales["payment"] = sales["payment"].fillna("Unknown")          # label,
we cannot guess the payment
med_median = sales.groupby("med_id")["unit_price"].transform("median")
sales["unit_price"] = sales["unit_price"].fillna(med_median)     # same
medicine -> same price level
print(sales.isna().sum())
```

```
sale_id          0
sale_date        0
pharmacy_city    0
med_id           0
quantity         0
unit_price       0
payment          0
customer_age     0
prescription     0
dtype: int64
```

In [21]:
```python
assert sales.isna().sum().sum() == 0
assert sales["quantity"].between(1, 50).all() and
sales["customer_age"].between(18, 100).all()
raw, clean = len(sales_raw), len(sales)
print(f"Raw rows: {raw} | Clean rows: {clean} | Removed: {raw - clean}
({(raw - clean) / raw * 100:.1f} %)")
```

```
Raw rows: 2440 | Clean rows: 2369 | Removed: 71 (2.9 %)
```

✅ **Why drop the invalid dates?** `sale_date` is a key field (monthly KPIs); there is no rule to rebuild it and only ~0.6 % of rows are concerned. **Why the median age?** The mean is pulled by extreme values; the median (middle value) is robust and keeps the distribution realistic. **Never 0**: an age or a price of 0 is fake data.

3.3 Analyse (4 pts)

1. Merge `sales` with `meds` (keep all the sales) into `df`. Create `revenue = quantity × unit_price` and `profit = revenue − quantity × cost_price`. Print total revenue, total profit and the global margin (%).
2. Revenue by city, sorted descending, with each city's **share of total revenue (%)**. Which city is first?
3. Top 5 medicines by revenue (name + revenue).
4. For each `category`: number of sales, revenue, profit and margin %. Which category is the most profitable? Which has the best margin?
5. Which **month** has the highest revenue? Compare the revenue of the *Cold & Flu* category in winter months (12, 1, 2) and summer months (6, 7, 8).
6. Average basket (`revenue`) per `payment` method, and the **% of sales with a prescription**.
7. Sales from **Tunis**, category **Antibiotics**, with `quantity >= 3`: count them with a boolean mask **and** with `query()` — same result?
8. Create `age_group` with `pd.cut`: `18-29`, `30-49`, `50-64`, `65+` and give the revenue per group.

In [22]:
```python
df = sales.merge(meds, on="med_id", how="left")
df["revenue"] = df["quantity"] * df["unit_price"]
df["profit"] = df["revenue"] - df["quantity"] * df["cost_price"]
total_rev, total_profit = df["revenue"].sum(), df["profit"].sum()
print(f"Revenue: {total_rev:,.2f} TND | Profit: {total_profit:,.2f} TND
| Margin: {total_profit / total_rev * 100:.1f} %")
```

```
Revenue: 109,627.93 TND | Profit: 49,173.43 TND | Margin: 44.9 %
```

In [23]:
```
city = df.groupby("pharmacy_city")
["revenue"].sum().sort_values(ascending=False).to_frame()
city["share_%"] = city["revenue"] / city["revenue"].sum() * 100
print("First city:", city.index[0])
city
```

```
First city: Tunis
```

Out[23]:

pharmacy_city	revenue	share_%
Tunis	25,971.21	23.69
Sfax	17,265.88	15.75
Sousse	15,031.54	13.71
Bizerte	11,294.21	10.30
Ariana	11,029.36	10.06
Gabès	10,818.10	9.87
Monastir	9,661.75	8.81
Nabeul	8,555.88	7.80

In [24]:
```
df.groupby("med_name")["revenue"].sum().nlargest(5)
```

Out[24]:
```
med_name
Blood pressure monitor   27,050.85
Insulin pen              10,587.35
Glucose test strips       9,074.26
Sunscreen SPF50           7,599.48
Baby diapers x40          6,960.54
Name: revenue, dtype: float64
```

In [25]:
```
cat = df.groupby("category").agg(sales=("sale_id", "count"), revenue=
("revenue", "sum"), profit=("profit", "sum"))
cat["margin_%"] = cat["profit"] / cat["revenue"] * 100
print("Most profitable:", cat["profit"].idxmax(), "| best margin:",
cat["margin_%"].idxmax())
cat.sort_values("profit", ascending=False)
```

```
Most profitable: Devices | best margin: Dermatology
```

Out[25]:

category	sales	revenue	profit	margin_%
Devices	282	41,308.17	18,965.17	45.91
Dermatology	313	17,346.97	8,917.67	51.41
Vitamins	398	9,834.58	5,001.98	50.86
Baby care	222	13,783.08	4,979.07	36.12
Diabetes	207	11,975.34	4,217.74	35.22
Antibiotics	268	8,158.24	3,577.84	43.86
Cold & Flu	288	3,898.89	1,960.69	50.29
Pain relief	391	3,322.66	1,553.26	46.75

In [26]:
```
df["month"] = df["sale_date"].dt.month
monthly = df.groupby("month")["revenue"].sum()
print("Best month:", monthly.idxmax(), f"({monthly.max():,.2f} TND)")
flu = df[df["category"] == "Cold & Flu"]
print("Cold & Flu winter:", round(flu[flu["month"].isin([12, 1, 2])]
["revenue"].sum(), 2),
      "| summer:", round(flu[flu["month"].isin([6, 7, 8])]
["revenue"].sum(), 2))
```

```
Best month: 12 (11,425.69 TND)
Cold & Flu winter: 2854.54 | summer: 334.21
```

In [27]:
```
print(df.groupby("payment")
["revenue"].mean().round(2).sort_values(ascending=False))
print(f"Sales with prescription: {(df['prescription'] == 'Yes').mean() *
100:.1f} %")
```

```
payment
Insurance (CNAM)   50.74
Card               47.14
Unknown            45.93
Cash               44.02
Name: revenue, dtype: float64
Sales with prescription: 40.9 %
```

In [28]:
```
mask = (df["pharmacy_city"] == "Tunis") & (df["category"] ==
"Antibiotics") & (df["quantity"] >= 3)
n_mask = len(df.loc[mask])
n_query = len(df.query("pharmacy_city == 'Tunis' and category ==
'Antibiotics' and quantity >= 3"))
print(n_mask, n_query, n_mask == n_query)
```

```
25 25 True
```

In [29]:
```
df["age_group"] = pd.cut(df["customer_age"], bins=[17, 29, 49, 64, 100],
labels=["18-29", "30-49", "50-64", "65+"])
df.groupby("age_group", observed=True)["revenue"].agg(["count",
"sum"]).round(2)
```

Out[29]:

age_group	count	sum
18-29	403	18,528.38
30-49	713	32,096.36
50-64	565	27,049.82
65+	688	31,953.39

3.4 Export and conclusion (1 pt)

1. Export `df` to `pharmacy_clean.csv` without the index, reload it with `parse_dates=["sale_date"]` and check that the shape is identical.
2. Write a **3-line conclusion** for the manager (data quality + 2 key insights).

In [30]:
```
df.to_csv("pharmacy_clean.csv", index=False)
check = pd.read_csv("pharmacy_clean.csv", parse_dates=["sale_date"])
print("Same shape:", check.shape == df.shape, check.shape, "|",
check["sale_date"].dtype)
```

```
Same shape: True (2369, 17) | datetime64[us]
```

✅ **Conclusion for the manager:** (1) *Data quality*: 2,440 raw lines → **2,369 trusted sales** (2.9 % removed: 40 duplicates, 14 invalid dates, 12 impossible quantities, 5 unknown medicines); missing prices, payments and ages were filled with documented rules. (2) 2025 revenue ≈ **109.6 k TND** for ≈ **49.2 k TND** profit (margin 44.9 %); **Tunis** brings ~24 % of revenue and **Devices** (blood-pressure monitors) are the main profit driver, while Dermatology has the best margin (51 %). (3) **Cold & Flu** sales are ~8× higher in winter than in summer → increase stock from November and run a summer promotion on Dermatology (sunscreen).